

Neural Networks for Data Science

Master's Degree in Data Science

Lecture 4: Fully-connected neural networks

Lecturer: S. Scardapane



SAPIENZA
UNIVERSITÀ DI ROMA

Feedforward neural networks

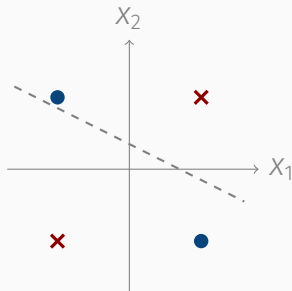
Basic definitions

For binary classification, a linear model separates the two classes using:

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Such a **linear boundary** can only separate relatively simple datasets.

Can we **learn a representation** of the data where a linear model is sufficient?



A simple non-linearly separable dataset (XOR).

A **fully-connected neural network**, also known as a **multilayer perceptron (MLP)**, learns an intermediate representation of the input:

Hidden representation

$$\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$

ϕ is an elementwise **activation function**.

Linear output

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{h} + c$$

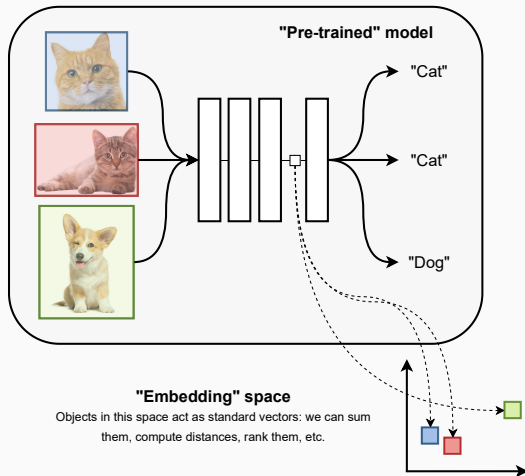
A linear model applied to the learned features $\mathbf{h}$.

The parameters $\mathbf{W}$, $\mathbf{b}$, $\mathbf{w}$, c are learned jointly from the data.

Without the nonlinearity ($\phi(z) = z$), the two affine maps collapse:

$$f(\mathbf{x}) = \mathbf{w}^\top (\mathbf{W}\mathbf{x} + \mathbf{b}) + c = (\mathbf{w}^\top \mathbf{W})\mathbf{x} + (\mathbf{w}^\top \mathbf{b} + c).$$

The hidden representation provides new coordinates for the input, in which a linear output layer can distinguish the classes.



Nothing prevents us from adding additional **hidden** (intermediate) layers:

$$\begin{aligned}\mathbf{h}^{(0)} &= \mathbf{x}, \\ \mathbf{h}^{(\ell)} &= \phi_{\ell}(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L, \\ f_{\boldsymbol{\theta}}(\mathbf{x}) &= \mathbf{w}^{\top} \mathbf{h}^{(L)} + c.\end{aligned}$$

All weights and biases belong to $\boldsymbol{\theta} = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L \cup \{\mathbf{w}, c\}$.

For widths $D_0, \dots, D_L$ (D_0 is the input dimension),

$$\mathbf{W}^{(\ell)} \in \mathbb{R}^{D_{\ell} \times D_{\ell-1}}, \quad \mathbf{b}^{(\ell)} \in \mathbb{R}^{D_{\ell}}.$$

Layer ℓ has $D_{\ell}(D_{\ell-1} + 1)$ parameters; the scalar output adds $D_L + 1$.

Widths and activations are layer-specific **hyperparameters**.

Given a dataset $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, we minimize:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i).$$

We use the losses introduced in Lecture 3, with the appropriate output for regression or classification (i.e., adding a softmax or sigmoid for multi-class or binary classification, respectively). In order to compute gradients we can use **automatic differentiation**.

Let $g : K \rightarrow \mathbb{R}$ be continuous on a compact set $K \subset \mathbb{R}^d$, and let ϕ be continuous and non-polynomial.

For any $\varepsilon > 0$, there exists an MLP with **one hidden layer**, sufficiently large width, and biases such that:

$$\sup_{\mathbf{x} \in K} |g(\mathbf{x}) - f_{\boldsymbol{\theta}}(\mathbf{x})| < \varepsilon.$$

This is an **existence result**. It does not tell us how much width or data we need, or whether training will find suitable parameters.

A single hidden layer can be sufficient for approximation; it need not be an efficient choice.

Representation / embedding

The feature vector $\mathbf{h}^{(\ell)}$ computed at an intermediate layer.

Width and depth

Width: $\dim \mathbf{h}^{(\ell)}$. Depth: L hidden layers ($L + 1$ with the output).

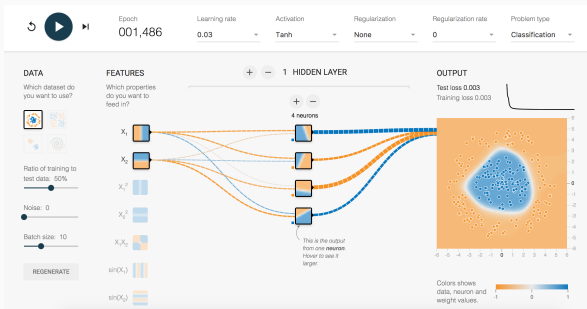
Parameters: learned

The weights and biases θ learned from the data.

Hyperparameters: chosen

Layer count, widths, activation functions, and learning rate.

Choosing hyperparameters is called **model selection / hyperparameter optimization**.



playground.tensorflow.org

Change the number and width of the hidden layers. How do the learned representations and the decision boundary change?

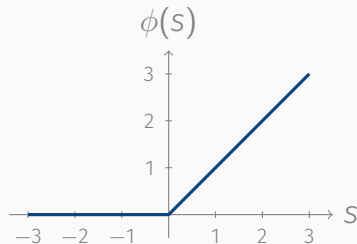
Feedforward neural networks

Activation functions

The **rectified linear unit** is defined as:

$$\phi(s) = \max(0, s).$$

In words: negative inputs become zero, positive inputs pass through unchanged.



ReLU is a good default choice for MLPs: we will explain more in the automatic differentiation lecture, when we study how gradients propagate through the network.

ReLU can be interpreted as a multiplication of the input by an on/off gate:

$$\text{ReLU}(s) = s g(s), \quad g(s) = \begin{cases} 0 & s < 0, \\ 1 & s \geq 0. \end{cases}$$

We can replace this hard decision with a **smooth weight** $g(s)$ between zero and one to obtain several interesting variants.



Dashed gray: hard gate

Solid blue: sigmoid $\sigma(s)$

Dashed green: Gaussian CDF $\Phi(s)$

SiLU (Sigmoid-weighted linear unit):

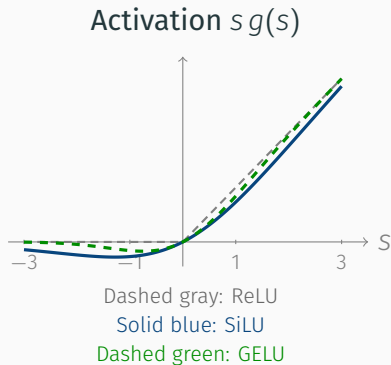
$$\text{SiLU}(s) = s \sigma(s), \quad \sigma(s) = \frac{1}{1 + e^{-s}}.$$

GELU (Gaussian error linear unit):

$$\text{GELU}(s) = s \Phi(s).$$

where Φ is the standard Gaussian cumulative distribution function.

Elfwing et al. (2017); Hendrycks and Gimpel (2016).



Instead of fixing the whole activation, we can let training adjust its shape.

$$\phi_{\alpha}(s) = \begin{cases} s & s \geq 0, \\ \alpha s & s < 0. \end{cases}$$

LeakyReLU

Fix a small negative-side slope, e.g.,
 $\alpha = 0.01$.

Parametric ReLU (PReLU)

Learn α jointly with the weights and biases.

For PReLU, α is part of θ . It can be shared or learned separately for different units.

He et al. (2015), Delving Deep into Rectifiers.

Learn a more flexible shape using fixed **basis functions**:

$$\phi(s) = \sum_{k=1}^K \alpha_k \psi_k(s).$$

The coefficients α_k are trained jointly with the network.

Kernel activation functions (KAFs)

$$\psi_k(s) = \kappa(s, d_k), \quad \kappa(s, d_k) = \exp(-\gamma(s - d_k)^2).$$

Example: Gaussian bumps with fixed centers d_k and width parameter $\gamma > 0$.

Non-parametric: a flexible basis expansion, still implemented with finitely many parameters. Splines are another option.

Scardapane et al. (2019), Kafnets.

We can also change where the learned scalar functions act:

MLP with learned activations

$$h_i = \phi_i \left(\sum_j W_{ij} x_j + b_i \right).$$

Mix inputs, then transform each unit.

KAN layer

$$h_i = \sum_j W_{ij} \phi_{ij}(x_j).$$

Transform each connection, then sum.

A KAN learns an activation at a unit, while a KAN learns functions on connections. Flexibility, however, comes with additional parameters and computation, making them less efficient than (equivalently deep) ReLU-based models.

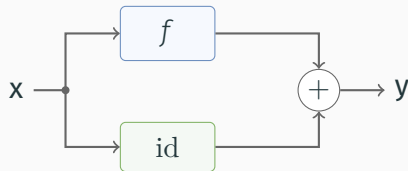
Liu et al. (2024), KAN: Kolmogorov–Arnold Networks.

Feedforward neural networks

Combining layers beyond the basic MLP

Two blocks can also process the same input in parallel. Given $\mathbf{u} = f(\mathbf{x})$ and $\mathbf{v} = g(\mathbf{x})$, we can combine their outputs in different ways:

- ▶ **Concatenation**, $[\mathbf{u}; \mathbf{v}]$, preserves both feature vectors and adds their widths.
- ▶ **Addition**, $\mathbf{u} + \mathbf{v}$, combines matching coordinates and requires equal shapes.
- ▶ Choosing the identity for one branch gives a **residual connection**: $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$.



Concatenation example: DenseNet (2017); residual networks: He et al. (2016).

Instead of adding branches with fixed coefficients, we can learn weights that depend on the input.

Highway connections

$$\mathbf{t} = \sigma(\mathbf{W}_t \mathbf{x} + \mathbf{b}_t),$$

$$\mathbf{y} = \mathbf{t} \odot f(\mathbf{x}) + (1 - \mathbf{t}) \odot \mathbf{x}.$$

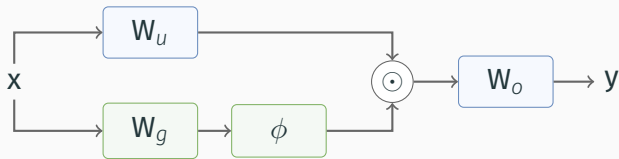
Each gate coordinate lies in $[0, 1]$ and controls how much of the input is carried forward. Similar gates appear in recurrent models; $\odot$ denotes elementwise multiplication.

Mixtures of experts (MoEs)

$$\mathbf{y} = \sum_{e=1}^E a_e(\mathbf{x}) f_e(\mathbf{x}).$$

A router weights the outputs of several experts with the same output shape. Sparse MoEs, used in language models, save computation by evaluating only the selected experts.

We can also multiply two branch outputs elementwise. In a GLU block, one projection determines how the other is weighted:



$$\mathbf{y} = \mathbf{W}_o [(\mathbf{W}_u \mathbf{x}) \odot \phi(\mathbf{W}_g \mathbf{x})] .$$

With $\phi = \sigma$ we obtain a **GLU**, whose gate lies in $[0, 1]$. Replacing it with SiLU gives **SwiGLU**, used in Llama: its multiplier can also be negative or exceed one.

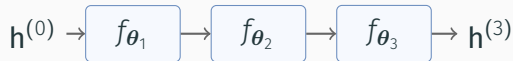
For $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{W}_u, \mathbf{W}_g \in \mathbb{R}^{M \times D}$ and $\mathbf{W}_o \in \mathbb{R}^{D \times M}$. Biases are omitted.

Shazeer (2020); Touvron et al., LLaMA (2023).

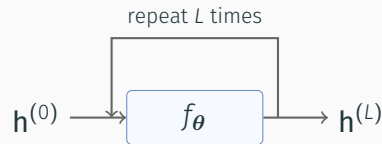
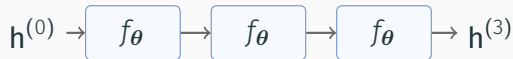
Sharing parameters and looping a block

A stack can use different parameters at each layer, or repeatedly apply the same block. The latter gives a **looped model**:

Separate parameters



Shared parameters

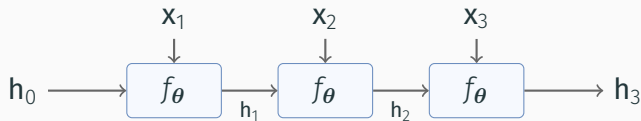


The shared block computes $\mathbf{h}^{(\ell+1)} = f_{\theta}(\mathbf{h}^{(\ell)})$, with matching input and output shapes. For a block with P parameters, sharing reduces the count from LP to P , but still requires L evaluations.

The unrolled example shows three applications. Dehghani et al., Universal Transformers (2019).

If we supply a new input at each application, the shared block becomes a **recurrent model**:

$$h_t = f_{\theta}(h_{t-1}, x_t).$$



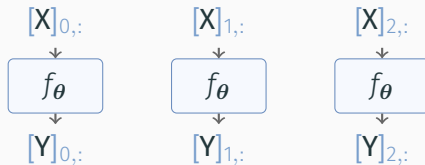
The hidden state carries information from earlier inputs, while the same parameters are used throughout the sequence. The model can therefore process sequences of different lengths.

Unlike a loop over depth, this recurrence advances through the input sequence at each step.

An MLP can process each position of a sequence $\mathbf{X} \sim (T, D)$ independently, using the same parameters:

$$[\mathbf{Y}]_{t,:} = f_{\theta}([\mathbf{X}]_{t,:}).$$

This mixes features within a position, but does not exchange information between positions.



The same idea applies to image pixels or patch vectors. Flattening the whole image instead allows global mixing, but ties the weights to a fixed layout.

Transformers use these **positionwise MLPs** alongside attention. Communication between positions requires another operation, such as convolution over neighborhoods, attention, or a recurrent state.